

ThreatLens

System Design & Database Development

Threat Detection & URL Scanner Platform

Version 1.0.0

Document Generated: September 07, 2026

Technologies: FastAPI · SQLite · SQLAlchemy · HTML/CSS/JS · Chrome Extension

Table of Contents

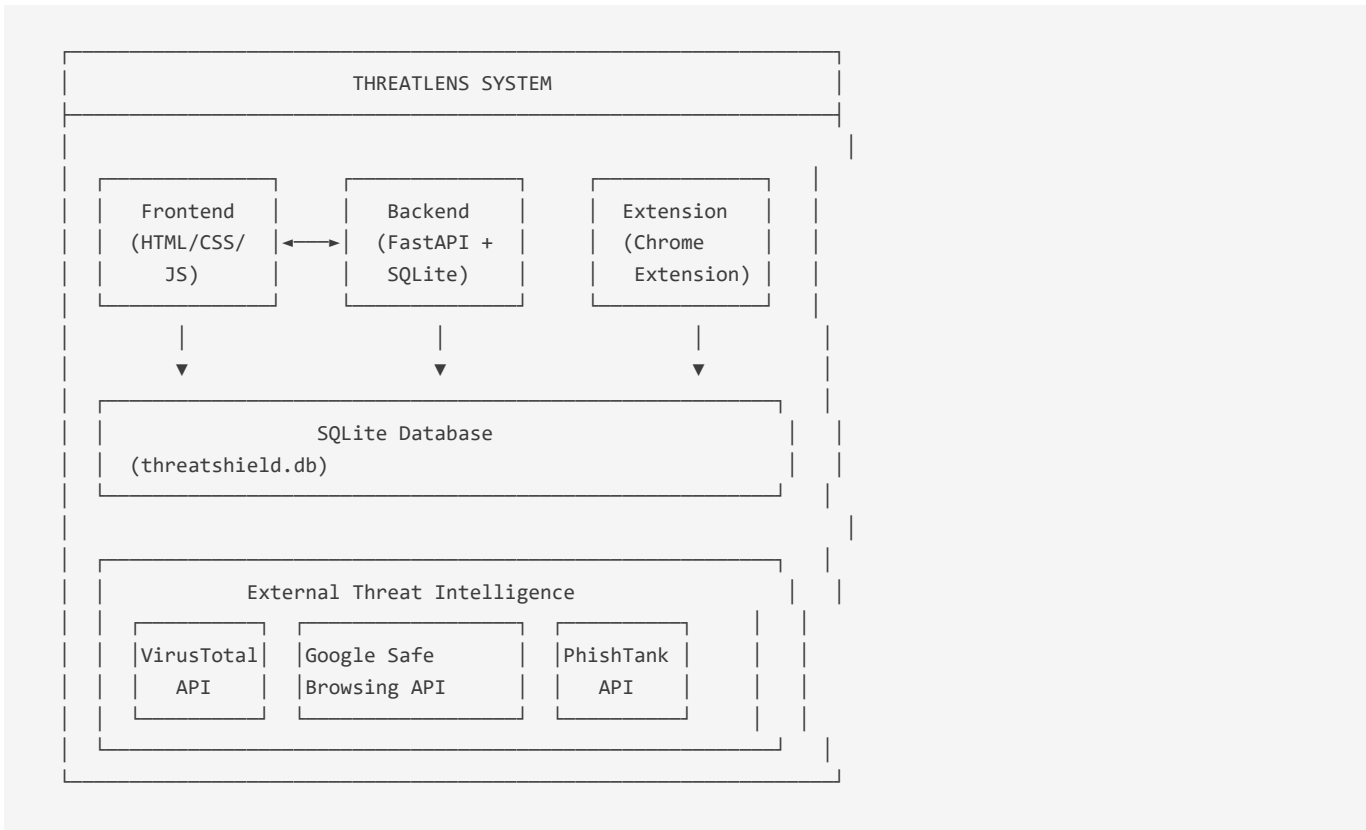
- 6. System Design & Database Development
- 7. UML Diagrams
 - 7.1 Use Case Diagram
 - 7.2 Class Diagram
 - 7.3 Activity Diagram
- 8. Data Dictionary
- 9. Finalized Database Schema

6. System Design & Database Development

6.1 System Architecture Overview

ThreatLens is a comprehensive threat detection platform consisting of three main components: a frontend dashboard (HTML/CSS/JS), a backend API server (FastAPI + SQLite), and a Chrome browser extension. The system integrates with external threat intelligence providers including VirusTotal, Google Safe Browsing, and PhishTank to deliver comprehensive URL and content security analysis.

Architecture Diagram



6.2 Technology Stack

Layer	Technology
Backend Framework	FastAPI (Python 3.11+)
Database	SQLite (via SQLAlchemy ORM)
Authentication	DB-backed bearer tokens (PBKDF)
Frontend	Vanilla HTML5, CSS3, JavaScript
External APIs	VirusTotal v3, Google Safe Bro
Extension	Chrome Extension Manifest V3
Rate Limiting	In-memory sliding window (per-

6.3 Backend Service Architecture

ROUTERS (API Endpoints):

auth /api/auth	scanner /api/scan	phishing /api/phish	admin /api/admin
users /api/users	history /api/hist	intel /api/intel	webhooks /api/web
audit /api/audit	tools /api/tool		

SERVICES (Business Logic):

scanner.py → Orchestrates full scan pipeline
heuristics.py → Typosquatting, TLD, entropy, structure checks
metadata.py → WHOIS, SSL, HTTP inspection
threat_intel.py → VirusTotal, Safe Browsing, PhishTank
risk_scoring.py → Unified 0-100 risk scoring engine
url_parser.py → URL normalization & parsing
phishing_analyzer.py → Rule-based NLP phishing detection
watchlist.py → User watchlist matching
security.py → Password hashing, token generation
rate_limit.py → Sliding window rate limiting

6.4 URL Scan Pipeline Flow

INPUT: Raw URL

|

▼

STEP 1: PARSE & NORMALIZE

- Add http:// if missing
- Extract: scheme, host, port, path, query
- Detect IP addresses (v4/v6)
- Extract: SLD, TLD, registrable domain
- Validate: only http/https allowed

▼

STEP 2: HEURISTIC ANALYSIS (offline, instant)

- Typosquatting detection (Levenshtein distance)
- Suspicious TLD check (.tk, .ml, .xyz, etc.)
- High entropy / randomized domain detection
- Structural checks: IP in URL, '@' deception, non-standard port, no HTTPS, excessive subdomains, URL shorteners, Punycode/IDN, credential paths

▼

STEP 3: METADATA INSPECTION (parallel via thread pool)

WHOIS	SSL	HTTP
- Age	- Issuer	- Status
- Registrar	- Valid	- Headers
- NS	- Days	- Chain
- Status	left	- Redirect

SSRF Protection: Block internal IPs

▼

STEP 4: THREAT INTELLIGENCE (optional, skip if no API keys)

- VirusTotal: Multi-engine AV scan + polling
- Google Safe Browsing: Threat list query
- PhishTank: Community phishing database check

▼

STEP 5: RISK SCORING

- Start: score = 100 (safe)
- Apply finding points (+/-) from all steps
- Clamp: 0-100
- Classify:
 - ≥85 → SAFE (green)
 - ≥70 → LOW RISK (yellow-green)
 - ≥40 → SUSPICIOUS (amber)
 - <40 → DANGEROUS (red)

▼

OUTPUT: JSON Risk Report

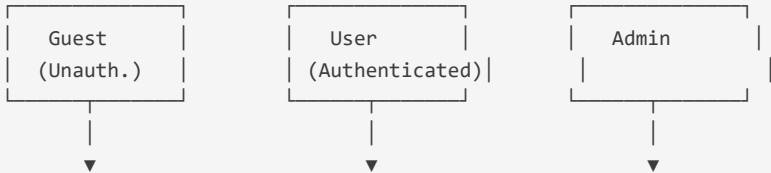
- risk_score, verdict, risk_level
- findings[], category_totals{}
- metadata{whois, ssl, http}
- threat_intel{vt, gsb, pt}
- duration_ms, scanned_at

7. UML Diagrams

7.1 Use Case Diagram

The Use Case Diagram illustrates the interactions between different user types (Guest, User, Admin) and the system functionalities.

ACTORS:



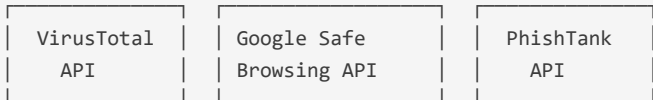
USER ACTIVITIES:

- Register Account
- Scan URL (Single/Bulk)
- Run Full Scan Pipeline (Parse → Heuristics → Metadata → Threat Intel → Risk Scoring)
- View Scan Results (Score, Verdict, Findings, Breakdown)
- Analyze Phishing Content
- Use Phishing Tools (Link Inspector, Email Headers, Sender Check, Watchlist)
- View History (Scans/Phishing Analyses)
- Search/Filter History (By Verdict, Date, Risk Level)
- Download Report (HTML/CSV/JSON)

ADMIN ACTIVITIES (Admin Only):

- View Platform Stats (Users, Scans, Detection Rate, etc.)
- List All Users
- Promote/Demote Users
- Delete Users
- View All Scans (Across All Users)
- View Audit Log
- Monitor Threat Intel Providers

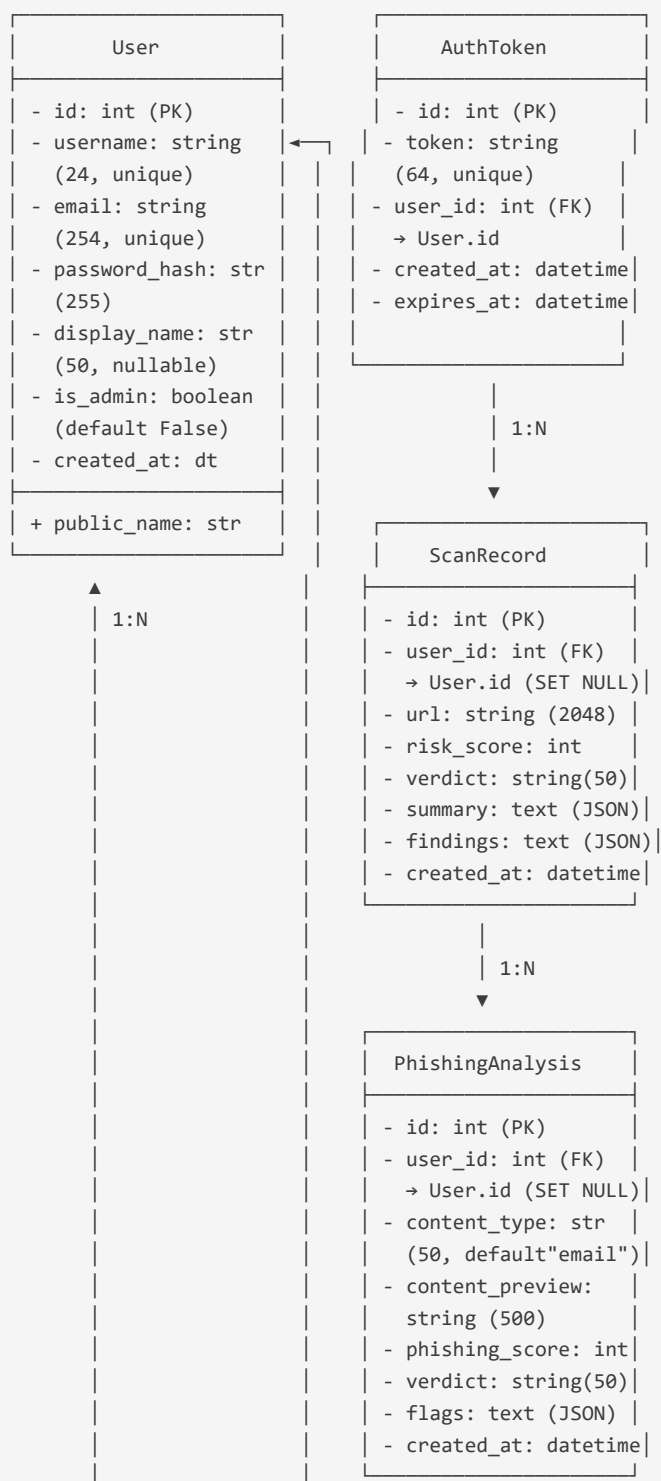
EXTERNAL SYSTEMS:



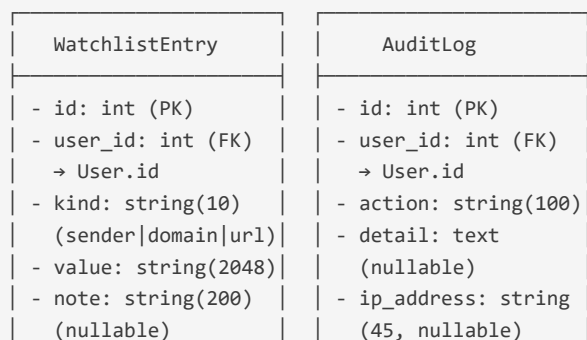
7.2 Class Diagram

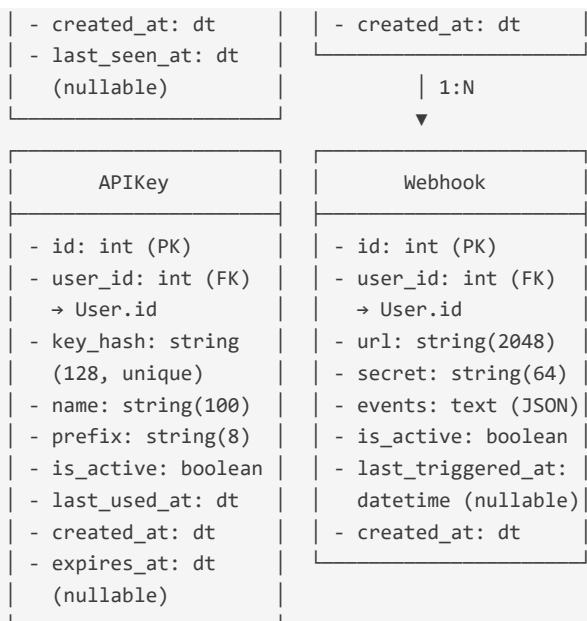
The Class Diagram shows the database entities (SQLAlchemy ORM models), service classes, frontend modules, and Chrome extension components with their relationships.

Entity Classes (SQLAlchemy Models)



ADDITIONAL ENTITIES:

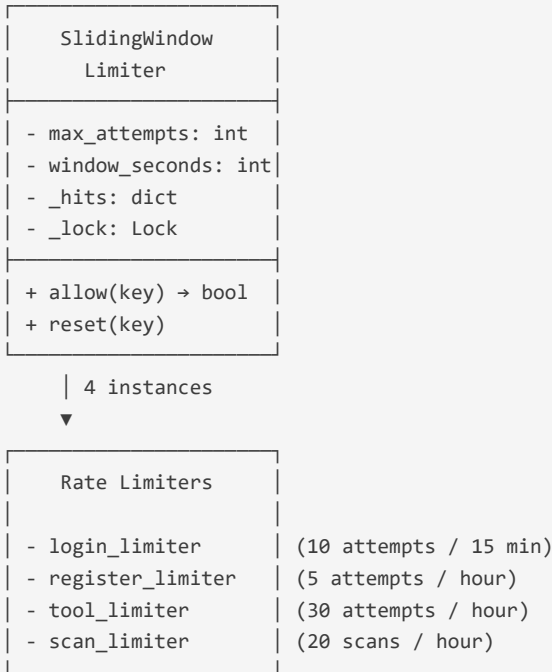


**RELATIONSHIPS:**

users 1 —> N auth_tokens
| —> N scans (ON DELETE SET NULL)
| —> N phishing_analyses (ON DELETE SET NULL)
| —> N watchlist_entries (ON DELETE CASCADE)
| —> N audit_log (ON DELETE SET NULL)
| —> N api_keys (ON DELETE CASCADE)
| —> N webhooks (ON DELETE CASCADE)

Service & Logic Classes

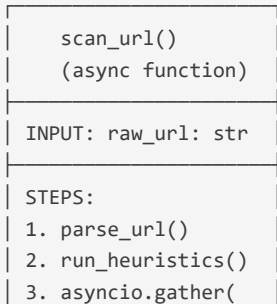
SLIDING WINDOW LIMITERS (rate_limit.py):



PARSED URL (url_parser.py - Dataclass):



SCAN ORCHESTRATION (scanner.py):



```
fetch_whois(),
fetch_ssl(),
fetch_http())
4. run_threat_intel()
5. compute_risk()
```

```
OUTPUT: dict with
- request, risk_score
- verdict, findings
- metadata, intel
- duration_ms
```

RISK SCORING (risk_scoring.py):

```
compute_risk()
```

```
INPUT: findings: list
of finding dicts
```

PROCESS:

- Start score: 100
- Apply each finding points (+/-)
- Clamp 0-100
- Classify verdict by threshold

OUTPUT: RiskReport

- score: int
- verdict: string
- risk_level: string
- findings: list
- category_totals

PHISHING ANALYZER (phishing_analyzer.py):

```
analyze_phishing()
```

```
INPUT: content: str
content_type
(email|sms|page)
```

PROCESS:

- Keyword detection (urgency,threat,financial,cred)
- Structural signals (greetings,sender,links,phone,etc)
- Score aggregation

OUTPUT: dict with

- phishing_score
- verdict
- confidence
- flags: list
- matched_keywords
- stats



Frontend JavaScript Modules

App (app.js - Singleton IIFE Module):

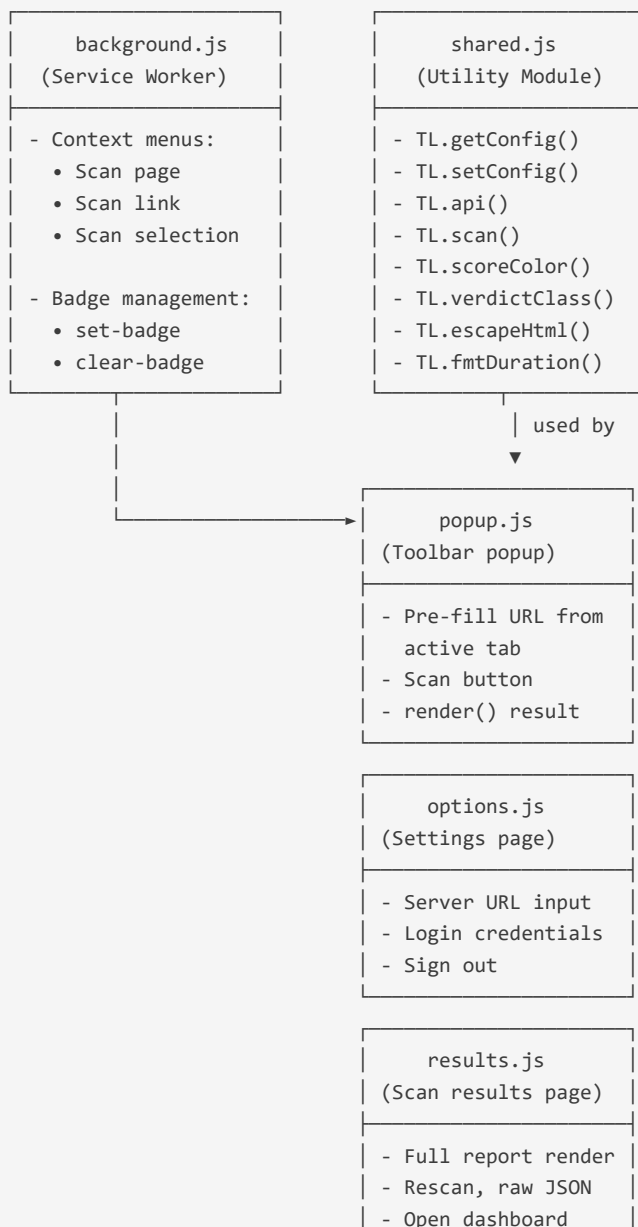
App (app.js)
STATE: - user: User null
API ENDPOINTS: - API.scan, scanBulk - API.auth (login/ register/logout/me) - API.phishing - API.history - API.intelFeed - API.stats, report - API.watchlist - API.admin
UTILITIES: - apiFetch() - toast() - escapeHtml() - verdictClass() - severityClass() - timeAgo() - initials() - animateValue()
BOOT FUNCTIONS: - boot() - renderShell() - renderNav() - loadUser() - checkApi() - ensureAdmin() - initTabs() - initKeyboardShort- cuts() - registerService- Worker()

PAGE MODULES (static/js/pages/):

scanner.js (scanner.html)	• runScan() / runBulkScan() • renderReport() (gauge, findings) • Compare mode, CSV import
auth.js (login/register.html)	• Login form handler • Register form handler • Token storage, admin flag
admin.js (admin.html)	• loadStats() / loadUsers() • loadScans() / loadAudit() • User promote/demote/delete
history.js (history.html)	• loadTab() (scans/phishing) • renderCards() with filtering • Verdict filter chips

intel.js (intel.html)	• loadProviders() / loadFeed() • Detection filter chips • Auto-refresh every 60s
learn.js (learn.html)	• Example library rendering • Spot-the-Phish quiz (5 questions)
phishing.js (phishing.html)	• Content Analyzer tab • renderPhishResult() (meter, flags)
phishing-tools.js	• Link Inspector • Email Header Analyzer • Sender Domain Check • Watchlist CRUD
profile.js (profile.html)	• loadProfile() / loadPhishTrends() • Report download (HTML/CSV/JSON) • Profile edit, password change

CHROME EXTENSION (extension/):



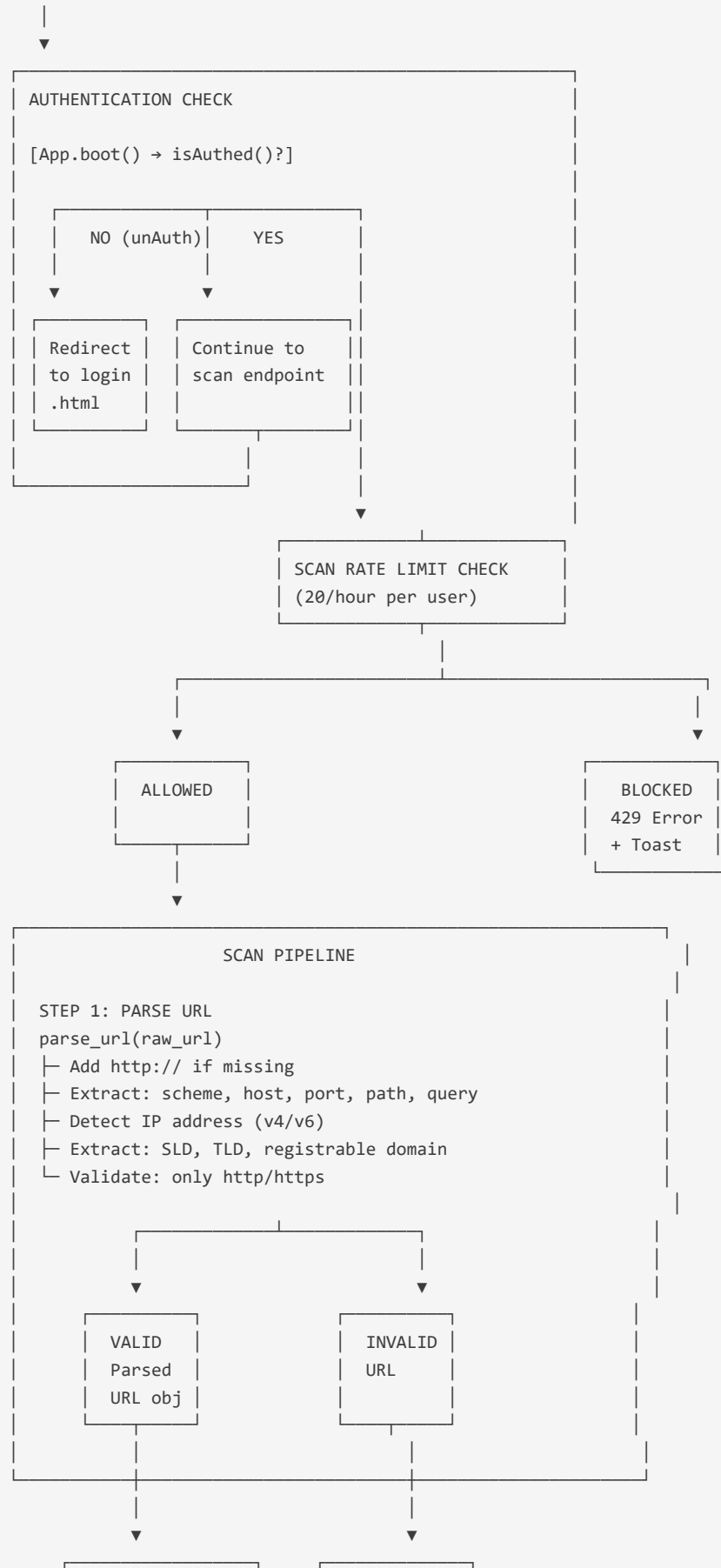


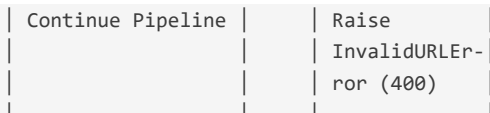
7.3 Activity Diagram

The Activity Diagram shows the step-by-step flow of a URL scan request through the system, including authentication, rate limiting, the scan pipeline stages, and the response rendering.

URL SCAN ACTIVITY FLOW:

START: User submits URL scan request





STEP 2: HEURISTIC ANALYSIS (offline, instant)
`run_heuristics(parsed) → list[findings]`

Checks:

- | `check_suspicious_tld()` (.tk, .ml, .xyz, etc.)
- | `check_high_entropy()` (randomized domain detection)
- | `check_typosquatting()` (Levenshtein distance to brands)
- | `check_structure()` (IP, '@', port, HTTPS, etc.)

STEP 3: METADATA INSPECTION (parallel via thread pool)

```

asyncio.gather(
    fetch_whois(parsed), ← WHOIS lookup
    fetch_ssl(parsed),   ← TLS certificate
    fetch_http(parsed),  ← HTTP + redirect chain
)
  
```

SSRF Guard: Each function blocks internal IPs

- Block: loopback, private, link-local, multicast
- Block: reserved, unspecified addresses

STEP 4: THREAT INTELLIGENCE (optional providers)

`run_threat_intel(parsed) → dict`

VirusTotal:

- | API key set?
- | YES → Scan URL + poll results
- | NO → Skip + note

Google Safe Browsing:

- | API key set?
- | YES → Query threat
- | NO → Skip + note

PhishTank:

- | API key set?
- | YES → Check URL in database
- | NO → Skip + note

STEP 5: RISK SCORING

All findings collected:

- `heuristic_findings` (from step 2)
- `metadata_findings` (from step 3)
- `intel_findings` (from step 4)

`compute_risk(all_findings) → RiskReport`

- Start: score = 100
- For each finding: score += points
- Clamp: `max(0, min(100, score))`
- Classify:

```
≥85 → SAFE (green)
≥70 → LOW RISK (yellow-green)
≥40 → SUSPICIOUS (amber)
<40 → DANGEROUS (red)
```

```
STEP 6: PERSIST TO DATABASE
INSERT INTO scans (user_id, url, risk_score, verdict,
                  summary, findings, created_at)
VALUES (?, ?, ?, ?, ?, ?, ?)

summary = JSON.stringify({
  request, category_totals, metadata,
  threat_intel, providers, duration_ms, scanned_at
})
findings = JSON.stringify(findings_array)
```

```
STEP 7: RETURN RESULT TO USER
Response: {
  scan_id: int,
  risk_score: int,
  verdict: string,
  risk_level: string,
  findings: [...],
  category_totals: {...},
  metadata: {whois, ssl, http},
  threat_intel: {vt, gsb, pt},
  duration_ms: int,
  scanned_at: ISO8601
}
```

```
FRONTEND: RENDER REPORT
renderReport(data) →
- Animate gauge (SVG circle)
- Display verdict badge
- Populate stat cards (age, TLS, HTTP, AV, etc.)
- Render findings list with severity colors
- Show category breakdown cards
- Populate metadata tab (WHOIS/SSL/HTTP details)
- Enable raw JSON view / download
```

END (Report shown)

ERROR PATHS:

- Timeout (504): "Scan timed out – the target may be slow"
- Scan failure: "Scan failed: {exception}"
- Invalid URL: "Invalid URL: {error message}"
- Rate limited: "Too many scan requests. Try again later."

Phishing Analysis Activity Flow

PHISHING ANALYSIS ACTIVITY FLOW:

START: User submits content for analysis

|



```
analyze_phishing(content, content_type)
```

KEYWORD DETECTION

```
URGENCY_KEYWORDS: "urgent", "immediately", "asap", ...
```

```
→ flag: "Urgency pressure" (+20 pts, high)
```

```
THREAT_KEYWORDS: "account suspended", "legal action",
```

```
... (16 terms)
```

```
→ flag: "Threats of consequences" (+25, critical)
```

```
FINANCIAL_KEYWORDS: "lottery", "you have won",
```

```
"tax refund", ... (16 terms)
```

```
→ flag: "Financial bait" (+20, high)
```

```
CREDENTIAL_KEYWORDS: "verify your account",
```

```
"reset password", ... (15 terms)
```

```
→ flag: "Credential harvesting" (+25, critical)
```

|



STRUCTURAL SIGNALS

```
- Generic greeting? (dear user, dear customer)
```

```
→ "Generic greeting" (+10, medium)
```

```
- Sender header check:
```

```
• Reply-To differs from From? (+15, high)
```

```
• Suspicious sender patterns (+15, high)
```

```
- Link checks:
```

```
• Links to IP address? (+15, high)
```

```
• URL shorteners? (+15, high)
```

```
- Anchor text mismatch:
```

```
"click here" → different host (+20, critical)
```

```
- Phone number present? (+5, low)
```

```
- BTC address present? (+15, high)
```

```
- Excessive CAPS (>70%)? (+5, low)
```

```
- Excessive emoji (>3)? (+5, low)
```

|



SCORE & VERDICT

```
raw_score = sum(all flag points)
```

```
phishing_score = min(100, raw_score)
```

```
IF score >= 55:
```

```
    verdict = "PHISHING"
```

```
    confidence = "high" if >= 75 else "medium"
```

```
ELIF score >= 25:  
    verdict = "SUSPICIOUS"  
    confidence = "medium"  
ELSE:  
    verdict = "SAFE"  
    confidence = "low"
```



```
PERSIST ANALYSIS TO DB  
  
INSERT INTO phishing_analyses  
(user_id, content_type, content_preview,  
 phishing_score, verdict, flags)  
VALUES (?, ?, ?, ?, ?, ?)
```



```
CHECK WATCHLIST MATCHES  
  
Extract sender domains + links from content  
match_watchlist(db, user_id, values)  
→ Return any matching watchlist entries  
→ Update last_seen_at on matches
```



```
RETURN RESULT  
{  
    analysis_id, phishing_score, verdict,  
    confidence, flags[], matched_keywords{},  
    stats{}, watchlist_matches[]  
}
```



```
END (render  
phishing meter  
+ flags)
```

8. Data Dictionary

The Data Dictionary provides a complete reference for all database tables, columns, data types, constraints, and enumerated values used in the ThreatLens system.

8.1 Core Tables

Table: users

COLUMN NAME	TYPE	NULLABLE	DEFAULT	DESCRIPTION
id	INTEGER (PK,	NOT NULL	AUTOINC	Primary key, auto-increment, indexed
username	STRING (24)	NOT NULL	—	Unique login name 3-24 chars [a-zA-Z0-9_] only Indexed, Unique
email	STRING (254)	NOT NULL	—	User email address RFC 5322 compliant Stored lowercase Indexed, Unique
password_hash	STRING (255)	NOT NULL	—	PBKDF2-HMAC-SHA256 format: pbkdf2_sha256\$salt\$ hex_digest 210,000 iterations
display_name	STRING (50)	NULLABLE	NULL	Optional friendly name for display
is_admin	BOOLEAN	NOT NULL	FALSE	Admin privilege flag (added post-launch)
created_at	DATETIME	NOT NULL	utcnow()	Account creation timestamp (UTC)

Table: auth_tokens

COLUMN NAME	TYPE	NULLABLE	DEFAULT	DESCRIPTION
id	INTEGER (PK, INDEXED)	NOT NULL	AUTOINC	Primary key
token	STRING (64)	NOT NULL	–	Bearer token (64 hex chars, random) Indexed, Unique
user_id	INTEGER (FK, INDEXED)	NOT NULL	–	FK → users.id On delete: CASCADE
created_at	DATETIME	NOT NULL	utcnow()	Token creation time (UTC)
expires_at	DATETIME	NOT NULL	–	Token expiry (30 days from creation)

Table: scans

COLUMN NAME	TYPE	NULLABLE	DEFAULT	DESCRIPTION
id	INTEGER (PK, INDEXED)	NOT NULL	AUTOINC	Primary key Auto-increment
user_id	INTEGER (FK, INDEXED)	NULLABLE	NULL	FK → users.id On delete: SET NULL Null = anonymous scan
url	STRING (2048)	–	–	Normalized URL (submitted URL) Indexed
risk_score	INTEGER	–	–	Risk score 0-100 (100 = safest)
verdict	STRING (50)	–	–	Classification: SAFE, LOW RISK, SUSPICIOUS, DANGEROUS
summary	TEXT	–	"{}"	JSON summary of scan
findings	TEXT	–	"[]"	JSON array of itemized findings
created_at	DATETIME	NOT NULL	utcnow()	Scan timestamp (UTC, Indexed)

Table: phishing_analyses

COLUMN NAME	TYPE	NULLABLE	DEFAULT	DESCRIPTION
id	INTEGER	NOT NULL	AUTOINC	Primary key
user_id	INTEGER (FK,	NULLABLE	NULL	FK → users.id On delete: SET NULL
content_type	STRING (50)	–	"email"	email sms page_text
content_preview	STRING (500)	–	""	First 400 chars of analyzed content
phishing_score	INTEGER	–	–	0-100 (higher = more likely phishing)
verdict	STRING (50)	–	–	SAFE, SUSPICIOUS, PHISHING
flags	TEXT	–	"[]"	JSON array of detection flags
created_at	DATETIME	NOT NULL	utcnow()	Analysis timestamp (UTC, Indexed)

Table: watchlist_entries

COLUMN NAME	TYPE	NULLABLE	DEFAULT	DESCRIPTION
id	INTEGER	NOT NULL	AUTOINC	Primary key
user_id	INTEGER (FK,	NOT NULL	—	FK → users.id On delete: CASCADE
kind	STRING (10)	NOT NULL	—	sender domain url
value	STRING (2048)	NOT NULL	—	Email address, domain, or URL
note	STRING (200)	NULLABLE	NULL	Optional user note (max 200 chars)
created_at	DATETIME	NOT NULL	utcnow()	When added (UTC)
last_seen_at	DATETIME	NULLABLE	NULL	Last matched (UTC)

Table: audit_log

COLUMN NAME	TYPE	NULLABLE	DEFAULT	DESCRIPTION
id	INTEGER	NOT NULL	AUTOINC	Primary key
user_id	INTEGER (FK,	NULLABLE	NULL	FK → users.id On delete: SET NULL
action	STRING (100)	NOT NULL	—	user.promote, user.demote, user.delete
detail	TEXT	NULLABLE	NULL	JSON or human- readable detail
ip_address	STRING (45)	NULLABLE	NULL	Client IP address (IPv4/IPv6)
created_at	DATETIME	NOT NULL	utcnow()	Action timestamp (UTC, Indexed)

Table: api_keys

COLUMN NAME	TYPE	NULLABLE	DEFAULT	DESCRIPTION
id	INTEGER	NOT NULL	AUTOINC	Primary key
user_id	INTEGER (FK,	NOT NULL	—	FK → users.id On delete: CASCADE
key_hash	STRING (128)	NOT NULL	—	Hashed API key (unique, indexed)
name	STRING (100)	—	"default"	Human label
prefix	STRING (8)	NOT NULL	—	First 8 chars (for display)
is_active	BOOLEAN	NOT NULL	TRUE	Currently active?
last_used_at	DATETIME	NULLABLE	NULL	Last used (UTC)
created_at	DATETIME	NOT NULL	utcnow()	Creation time (UTC)
expires_at	DATETIME	NULLABLE	NULL	Optional expiry

Table: webhooks

COLUMN NAME	TYPE	NULLABLE	DEFAULT	DESCRIPTION
id	INTEGER	NOT NULL	AUTOINC	Primary key
user_id	INTEGER (FK,	NOT NULL	—	FK → users.id On delete: CASCADE
url	STRING (2048)	NOT NULL	—	Webhook endpoint (HTTP/S only)
secret	STRING (64)	NOT NULL	—	HMAC-SHA256 secret (32 hex chars)
events	TEXT	—	'["scan. dangerous"]' '	JSON array of event types: scan. dangerous, scan.*
is_active	BOOLEAN	NOT NULL	TRUE	Currently active?
last_triggered	DATETIME	NULLABLE	NULL	Last triggered (UTC)
created_at	DATETIME	NOT NULL	utcnow()	Creation (UTC)

8.2 Enumerated Values Reference

VERDICT VALUES (ScanRecord.verdict, PhishingAnalysis.verdict):

- SAFE — URL/content appears legitimate
- LOW RISK — Minor concerns, generally safe (scans only)
- SUSPICIOUS — Notable red flags, warrants caution
- DANGEROUS — High-risk URL (scan verdict only)
- PHISHING — Likely phishing content (phishing only)

RISK LEVEL VALUES (RiskReport.risk_level):

- safe — Score ≥ 85
- low — Score ≥ 70 and < 85
- suspicious — Score ≥ 40 and < 70
- critical — Score < 40

WATCHLIST ENTRY KIND VALUES:

- sender — Email address (e.g. scammer@gmail.com)
- domain — Domain name (e.g. paypal-verify.tk)
- url — Full URL (e.g. https://fake-login.xyz)

CONTENT TYPE VALUES:

- email — Email body content
- sms — SMS/text message content
- page_text — Web page text content

FINDING SEVERITY VALUES:

- critical — Severe security issue
- high — Significant concern
- medium — Moderate concern
- low — Minor concern
- info — Informational only (no score impact)
- positive — Positive signal (increases score)

THREAT INTEL STATUS VALUES:

- ok — Provider query succeeded
- skipped — API key not configured
- error — Provider query failed

AUTH ACTION VALUES (AuditLog.action):

- user.promote — User promoted to admin
- user.demote — User demoted from admin
- user.delete — User deleted

WEBHOOK EVENT TYPES:

- scan.dangerous — Fired when scan returns DANGEROUS verdict
- * — Wildcard (all events)

8.3 Reference Data (Hardcoded)

SUSPICIOUS TLDs (57 high-risk TLDs):

tk, ml, ga, cf, gq, top, xyz, work, click, loan, men, win, bid, date, racing, review, stream, download, gdn, country, kim, science, party, link, wang, xin, monster, rest, fit, accountant, trade, website, site, online, live, club, cyou, icu, buzz, quest, cam, mom, lol, zip, mov

URL SHORTENERS (21 known services):

bit.ly, tinyurl.com, t.co, goo.gl, ow.ly, is.gd, buff.ly, rebrand.ly, cutt.ly, shorturl.at, rb.gy, tiny.cc, s.id, v.gd, soo.gd, t.ly, lnkd.in, x.co, qr.ae, cutt.us

POPULAR BRANDS (56 brands with official domains):

google (google.com), youtube (youtube.com), gmail (gmail.com), facebook (facebook.com), instagram (instagram.com), whatsapp (whatsapp), twitter (twitter.com, x.com), telegram (telegram.org), tiktok, snapchat, paypal (paypal.com), amazon (amazon.com), ebay (ebay.com), apple (apple.com), icloud (icloud.com), microsoft (microsoft.com), outlook (outlook.com), office (office.com), onedrive (onedrive.com), netflix (netflix.com), spotify (spotify.com), twitch (twitch.tv), linkedin (linkedin.com), github (github.com), dropbox (dropbox.com), steam (steampowered.com), epicgames (epicgames.com), coinbase (coinbase.com), binance (binance.com), blockchain (blockchain), metamask (metamask.io), chase (chase.com), wells Fargo (wellsfargo.com), bankofamerica (bankofamerica.com), citibank (citi.com), capitalone (capitalone.com), hsbc (hsbc.com), barclays (barclays.co.uk), fedex (fedex.com), ups (ups.com), usps (usps.com), dhl (dhl.com), walmart (walmart.com), target (target.com), bestbuy (bestbuy.com), airbnb (airbnb.com), booking (booking.com), expedia (expedia.com), adobe (adobe.com), salesforce (salesforce.com), wordpress (wordpress.com)

CREDENTIAL HINTS (15 keywords):

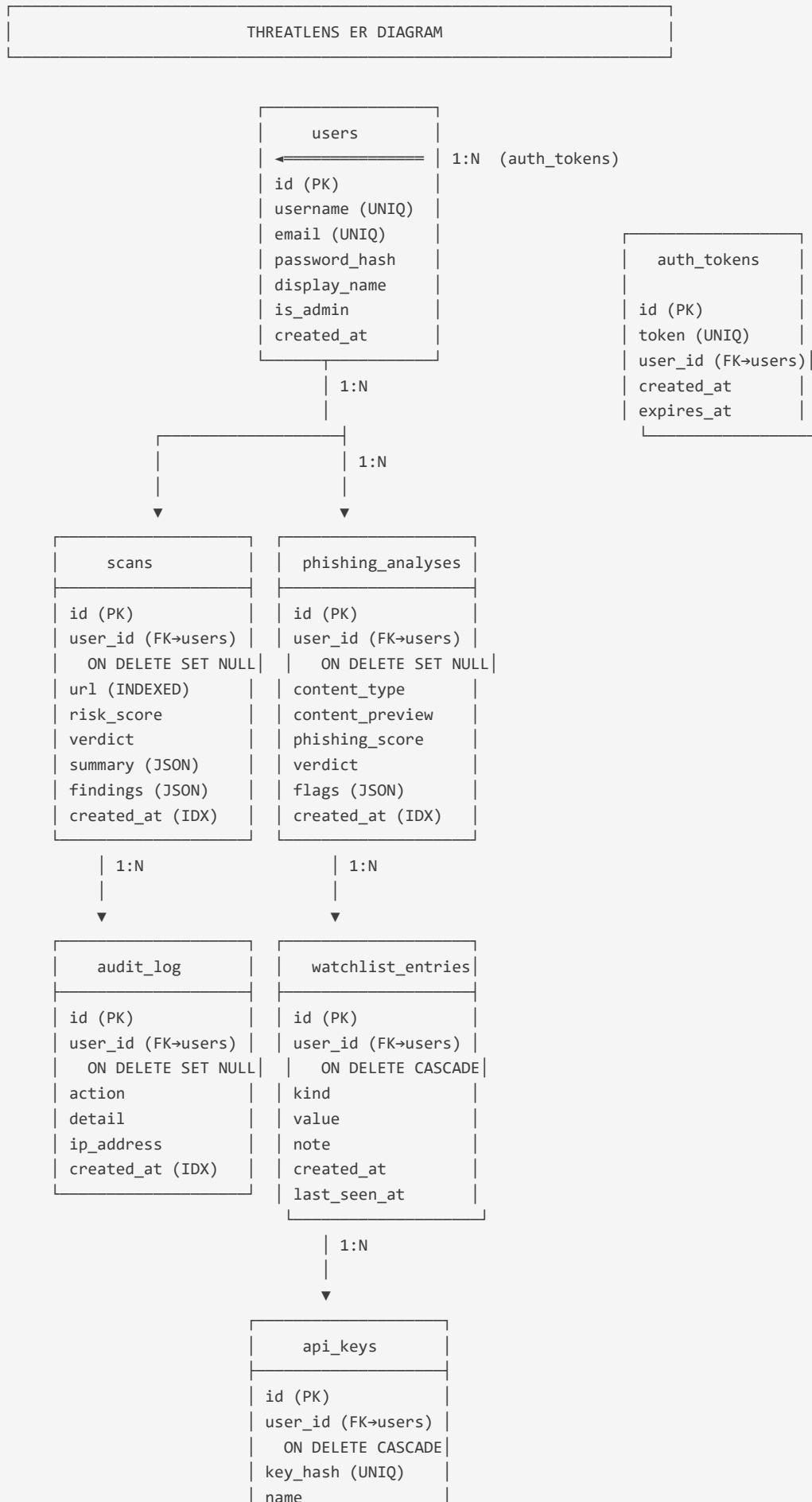
login, signin, verify, secure, account, update, confirm, wallet, auth, password, recover, unlock, validate, billing

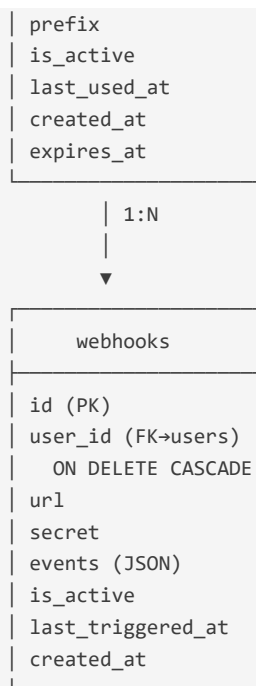
FREE MAIL DOMAINS (15 providers):

gmail.com, googlemail.com, yahoo.com, yahoo.co.uk, hotmail.com, outlook.com, live.com, aol.com, icloud.com, protonmail.com, proton.me, mail.com, gmx.com, zoho.com

9. Finalized Database Schema

9.1 Complete Entity-Relationship Diagram





RELATIONSHIPS SUMMARY:

users 1 —→ N auth_tokens (A user has many tokens)
 |→ N scans (A user has many scans)
 |→ N phishing_analyses (A user has many phishing analyses)
 |→ N watchlist_entries (A user has many watchlist items)
 |→ N audit_log (A user generates audit entries)
 |→ N api_keys (A user has many API keys)
 |→ N webhooks (A user has many webhooks)

CASCADE vs SET NULL:

- CASCADE: Delete user → delete their tokens, watchlist, API keys, webhooks (dependent data)
- SET NULL: Delete user → keep scans/analyses/audit but anonymize (historical record preservation)

9.2 Index Strategy

INDEX STRATEGY		
Table	Columns Indexed	Purpose
users	id (PK)	Primary lookup
	username (UNIQUE)	Login lookup
	email (UNIQUE)	Login by email
auth_tokens	id (PK)	Primary lookup
	token (UNIQUE)	Token validation
	user_id	User's tokens lookup
scans	id (PK)	Primary lookup
	url	URL search/filter
	user_id	User's scans lookup
	created_at	Chronological ordering
phishing_analyses	id (PK)	Primary lookup
	user_id	User's analyses lookup
	created_at	Chronological ordering
watchlist_entries	id (PK)	Primary lookup
	user_id	User's watchlist lookup
audit_log	id (PK)	Primary lookup
	user_id	User's actions lookup
	created_at	Chronological ordering
api_keys	id (PK)	Primary lookup
	key_hash (UNIQUE)	Key validation
	user_id	User's keys lookup
webhooks	id (PK)	Primary lookup
	user_id	User's webhooks lookup

9.3 Schema Evolution History

INITIAL SCHEMA (v1.0):

- users (id, username, email, password_hash, display_name, created_at)
- auth_tokens (id, token, user_id, created_at, expires_at)
- watchlist_entries (id, user_id, kind, value, note, created_at, last_seen_at)
- scans (id, url, risk_score, verdict, summary, findings, created_at)
- phishing_analyses (id, content_type, content_preview, phishing_score, verdict, flags, created_at)
- audit_log (id, user_id, action, detail, ip_address, created_at)

SCHEMA UPDATE (v1.1+ - _ensure_legacy_columns in main.py):

- Added users.is_admin (BOOLEAN DEFAULT 0) – added via ALTER TABLE for existing databases (new DBs get it from ORM metadata)
- Added scans.user_id (INTEGER, FK to users.id ON DELETE SET NULL)
- Added phishing_analyses.user_id (INTEGER, FK to users.id ON DELETE SET NULL)

WHY user_id uses SET NULL:

Historical scans/analyses are preserved even if a user deletes their account. The record remains but is disassociated from the deleted user.

WHY watchlist/api_keys/webhooks use CASCADE:

These are strictly user-owned – if the user is deleted, their watchlist entries, API keys, and webhook subscriptions should be removed.

9.4 JSON Column Structures

SCAN.summary JSON Structure:

```
{
  "request": {
    "submitted_url": "https://example.com",
    "normalized_url": "https://example.com/",
    "scheme": "https",
    "host": "example.com",
    "port": 443,
    "path": "/",
    "is_ip": false,
    "ip_version": null,
    "domain": "example",
    "tld": "com",
    "registrable_domain": "example.com",
    "subdomain": "",
    "url_length": 22,
    "has_https": true
  },
  "category_totals": {
    "Heuristics": -20,
    "Metadata": -10,
    "Threat Intelligence": -50
  },
  "metadata": {
    "whois": { ... },
    "ssl": { ... },
    "http": { ... }
  },
  "threat_intel": {
    "virustotal": { "status": "ok", "malicious": 5, ... },
    "google_safe_browsing": { "status": "ok", "threatened": true, ... },
    "phishtank": { "status": "ok", "in_database": false }
  },
  "providers": {
    "virustotal": true,
    "google_safe_browsing": true,
    "phishtank": true
  },
  "duration_ms": 3420,
  "scanned_at": "2026-09-07T12:00:00.000000+00:00"
}
```

SCAN.findings JSON Structure (array):

```
[
  {
    "label": "Typosquatting / brand impersonation",
    "points": -35,
    "severity": "critical",
    "detail": "'example' is 1 character away from 'exampel'",
    "category": "Heuristics"
  },
  {
    "label": "Domain registered very recently",
    "points": -40,
    "severity": "critical",
    "detail": "Domain is 3 days old (< 14 days)",
    "category": "Metadata"
  }
]
```

```
PHISHING_ANALYSES.flags JSON Structure (array):
[
  {
    "label": "Urgency pressure",
    "points": 20,
    "severity": "high",
    "detail": "Uses urgency language: urgent, immediately"
  },
  {
    "label": "Credential harvesting language",
    "points": 25,
    "severity": "critical",
    "detail": "Asked to verify/enter credentials: verify your account"
  }
]

WEBHOOKS.events JSON Structure (array):
["scan.dangerous", "scan.*"]
```

9.5 Database Configuration

```
# backend/app/config.py → Settings
DATABASE_URL: str = os.getenv("DATABASE_URL", "sqlite:///./threatshield.db")

# backend/app/database.py
engine = create_engine(
    settings.DATABASE_URL,
    connect_args={"check_same_thread": False} if sqlite else {},
    pool_pre_ping=True
)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = DeclarativeBase() # SQLAlchemy 2.0 style
```

9.6 Key Design Decisions

KEY DESIGN DECISIONS	
Decision	Rationale
SQLite over PostgreSQL	Single-file, zero-config, suitable for small-medium deployments. Easy to backup (copy file).
SQLAlchemy 2.0 ORM	Type-safe, async-ready, declarative models. Future migration to PostgreSQL just requires changing DATABASE_URL.
Bearer tokens in DB (not JWT)	Simple revocation (DELETE from auth_tokens). No clock skew issues. 30-day TTL.
PBKDF2-HMAC-SHA256 (stdlib only)	No external crypto dependencies. 210,000 iterations is strong against brute force.
SET NULL on user deletion for scans	Preserve historical data. Scans remain visible but anonymized.
CASCADE on user deletion for tokens/watchlist/keys/webhooks	These are purely user-owned – if user deleted, their watchlist, keys, webhooks removed.
JSON columns for summary/findings/flags	Flexible schema for evolving report structure without migrations.
In-memory rate limiter	Simple, sufficient for single-worker. Documented to move to Redis for multi-worker.
SSRF protection in metadata.py	Scanner must never be an internal network proxy. All outbound connections validated.